PRD: CoreBanking Sandbox

1. Header

Field	Value
Product	CoreBanking Sandbox
ID/Track	PST_PD_153
Batch / Level	2026 Level 2 (Core)
Duration	4 weeks
Student/Team	Kunal Tayde, Gajangi Nikhileshwar (2-student team)
PRD version	v0.1

2. Overview

One-month OJT demo of CoreBanking Sandbox for FinTech Developers: a toy banking sandbox with accounts, transfer and statement. Developers create sample bank accounts, move money between them with validated transfers, and view a statement with a running balance for each account. Balances can never go negative unless an account has an overdraft flag. Java/Go core-banking stacks are skipped; the demo uses HTML, CSS, JavaScript and Bootstrap 5 with a small Python 3 helper, and data is saved locally so every transaction can be reviewed later.

- **Industry:** FinTech
- **Theme:** Banking Systems
- **Product promise:** A developer can create two accounts, transfer money between them, and see both statements update with correct balances in under 30 seconds.
- **Primary objective:** Give FinTech developers a safe, simple place to understand and test core banking rules (account creation, transfers, balance checks and statements) without a real bank or a heavy core-banking stack.
- **Demo limitation:** All accounts, customers and amounts are synthetic. No real money moves, and the sandbox does not meet banking security, regulatory or accounting standards.

3. Problem statement

Developers learning or building FinTech features often have no safe place to try basic banking flows. Real bank sandboxes need sign-ups and API keys, and full core-banking systems in Java or Go are far too large to understand in a month. Without a simple model, it is easy to write transfers that create money, allow negative balances, or produce statements that do not add up.

CoreBanking Sandbox addresses this with a toy banking sandbox that shows accounts, transfers and statements in one small app, with clear rules and visible results.

- **Primary problem and user pain point:** A developer testing a payment feature transfers money and gets a negative balance with no warning, then cannot tell from the scattered logs

which transfer caused it. The moment of frustration is staring at a balance that does not match the list of transactions and not knowing where the numbers went wrong.

- **Research questions:**
    - Can a developer create accounts and complete a transfer without reading the code?
    - Do clear validation messages help users understand why a transfer was blocked?
    - Does a statement with opening balance, running balance and closing balance make it easy to check that the numbers add up?
- **Hypothesis:** A small sandbox with strict transfer rules and a running-balance statement helps developers understand core banking behaviour faster than reading documentation alone.
- **Decision the product supports:** Whether a transfer should be allowed or blocked, and whether an account's statement balances correctly against its transactions.

4. Personas

Persona	Primary need	Expected benefit
FinTech Developer	Try account, transfer and statement flows safely with sample data	Clear understanding of banking rules and quick testing
Sandbox Admin / Team Lead	Set up accounts, manage overdraft flags and reset test data	Controlled test environment with visible rules

Contextual persona cards

- **FinTech Developer:** Student or junior developer learning how banking apps work, comfortable with a laptop and browser tools, sometimes checking results on a phone. *Goals:* create test accounts, make transfers, check statements, and see how errors are handled. *Frustrations:* complex bank APIs, negative balances with no explanation, statements that do not match transactions. *Workflow:* open the dashboard, create or pick accounts, make a transfer, read the result, open the statement.
- **Sandbox Admin / Team Lead:** Senior developer or mentor who prepares the sandbox for a team or class, usually on a laptop. *Goals:* load sample accounts, decide which accounts may use an overdraft, and reset data before a demo. *Frustrations:* test data changed by others, unclear account limits, no quick way to start fresh. *Workflow:* switch to the Admin view, create accounts, set overdraft flags and limits, review all accounts, reset sample data.

5. Key user journeys

Journey 1: Create an account

1. The user opens CoreBanking Sandbox and goes to Accounts.
2. The user enters the account holder name, account type (savings or current) and an opening deposit.
3. The form validates the fields; the app generates an account number and saves the account.

4. The new account appears in the account list with its balance and status Active.

Journey 2: Transfer money

1. The user opens Transfer and selects a source account and a destination account.
2. The user enters an amount and an optional reference note.
3. The app checks the rules: different accounts, amount greater than 0, and enough balance unless the source account has an overdraft flag.
4. If allowed, the app records a debit and a credit with the same transfer ID and shows a success message with the new balances.
5. If blocked, the app explains the reason and no balance changes.

Journey 3: View a statement

1. The user opens Statement and selects an account.
2. The statement table lists each transaction with date, description, debit, credit and running balance.
3. The user filters by date range or transaction type.
4. The user checks the opening and closing balances and exports the statement as CSV (good to have).

6. Scope

- **Must have (MVP - graded):**

- Create account: holder name, account type, opening deposit, generated account number.
- Transfer: move money between two accounts with validation, recording a matching debit and credit.
- Statement table: per-account list of transactions with running balance, opening and closing balance.
- No negative balance without overdraft flag: transfers that would overdraw are blocked unless the account has an overdraft flag and stays within its limit.
- localStorage (or JSON file) persistence with sample data, and a responsive Bootstrap layout (laptop + phone-width).

- **Good to have:**

- CSV export of the statement table.
- Search and filters on accounts and statements.
- Simple Developer / Admin view switch.

- **Stretch (optional self-study):**

- Hash-chained ledger (DSA): each ledger entry stores the hash of the previous entry so any edit to past entries can be detected.
- React + Vite interface, Chart.js balance chart, or a tiny Python Flask / JSON-file helper.

- **Out of scope:**

- Real money, payment rails (UPI, NEFT, card networks) or connections to real banks.
- KYC, anti-money-laundering checks, real customer data, login and server-side security.
- Interest, loans, fees, multi-currency exchange and full Java/Go core-banking stacks.

7. User stories

ID	User story	Acceptance criteria
US-01	As a developer, I want to create an account so that I can test banking flows.	Name, type and an opening deposit of 0 or more are required; a unique account number is generated.
US-02	As a developer, I want to see all accounts with balances so that I know the current state of the sandbox.	Account list shows number, holder, type, balance, overdraft flag and status.
US-03	As a developer, I want to transfer money between accounts so that I can test payments.	Amount must be greater than 0 and accounts must differ; a debit and a credit with the same transfer ID are recorded.
US-04	As a developer, I want transfers that would overdraw an account to be blocked so that balances stay valid.	Without an overdraft flag, a transfer above the balance is blocked with a clear reason and no balance changes.
US-05	As an admin, I want to set an overdraft flag and limit so that selected accounts can go negative safely.	Only the Admin view can change the flag; balance can go negative only down to the set limit.
US-06	As a developer, I want a statement for each account so that I can check every transaction.	Statement shows date, description, debit, credit and running balance; closing balance equals the account balance.
US-07	As a developer, I want to filter and export a statement so that I can review or share it.	Date and type filters narrow rows; CSV downloads with the visible rows (good to have).
US-08	As an admin, I want to reset the sandbox so that I can start a demo with clean sample data.	Reset asks for confirmation and restores the documented sample accounts and transactions.
US-09	As a developer, I want admin-only controls hidden in my view so that test settings are not changed by mistake.	Overdraft and reset controls are not shown in the Developer view; a direct attempt shows an unauthorized message.
US-10	As a user, I want the sandbox to work on phone and laptop so that I can check results anywhere.	Forms, account list and statement stay usable at phone-width and laptop-width.

8. Screen inventory

Screen	Key elements	Primary action
Dashboard	Total accounts, total balance, recent transfers, role switch	Review sandbox status
Accounts	Account list with number, holder, type, balance, overdraft flag; search; Create Account button	Find or create an account
Create Account	Holder name, account type, opening deposit, validation messages	Create a new account
Transfer	From and to account selectors, amount, reference, rule hints, Transfer button	Transfer money
Statement	Account selector, opening balance, transaction table with running balance, closing balance, filters, export	Review transactions
Admin Settings	Overdraft flag and limit per account, reset sample data	Manage sandbox rules
About / Help	Banking rules, sample data notes, demo limits	Learn how the sandbox works

9. Component breakdown

Component	Responsibility	Key inputs / events
RoleSwitch	Switch between Developer and Admin views	Role, change
SummaryCards	Show account count, total balance and recent activity	Accounts, transactions
AccountForm	Collect and validate new account details	Fields, submit, errors
AccountList	List accounts with balance and flags	Accounts, search, select
TransferForm	Collect transfer details and show the result	From, to, amount, submit
TransferService	Apply transfer rules and record debit and credit entries	Transfer request; returns result or error
BalanceCalculator	Compute balances and running balances from transactions	Account, transactions
StatementTable	Show transactions with running balance and totals	Account, rows, filters
OverdraftSettings	Edit overdraft flag and limit (admin only)	Account, flag, limit, save
RoleGuard	Show or hide admin controls based on role	Role, requested action
ExportButton	Download visible statement rows as CSV	Rows, click

Component	Responsibility	Key inputs / events
FeedbackMessage	Accessible success, warning and error messages	Message, type, dismiss
LocalDataStore	Read and write data using documented keys	Load, save, reset
bank.py (Python helper)	Account and Bank classes with the same rules, reading and writing the JSON data file for a CLI demo	Commands: create, transfer, statement

10. State and data

Core data entities

Entity	Important fields	Purpose
Account	accountId, accountNumber, holderName, type, openingDeposit, overdraftEnabled, overdraftLimit, status, createdAt	One sample bank account
Transaction	txnId, transferId, accountId, type (debit/credit), amount, reference, balanceAfter, createdAt	One ledger entry for an account
Transfer	transferId, fromAccountId, toAccountId, amount, reference, status, reason, createdAt	One transfer attempt and its result
Preferences	role, lastAccountId, statementFilters	Stores non-sensitive UI choices

State management

- **Persistence keys (localStorage):** corebank_accounts, corebank_transactions, corebank_transfers, corebank_prefs.
- **Python data file:** bank_data.json with the same shape as the localStorage keys, used by the Python CLI helper.
- **Derived data:** current balance per account, running balance per statement row, opening and closing balance for a date range, total money in the sandbox, available balance including overdraft, and filtered rows.
- **Transfer rule:** amount greater than 0; source and destination are different active accounts; new source balance must be 0 or more, or no lower than minus the overdraft limit when the overdraft flag is on; debit and credit are saved together or not at all.
- **Money rule:** amounts are stored as whole numbers in paise to avoid rounding errors and shown with two decimals.
- **UI-only state:** active screen, current role, selected account, form values and errors, filters, submitting flag, and confirmation dialog state.
- **Sample data:** five sample accounts (one with an overdraft flag) and about 20 transactions so every screen has data on first load. All data is synthetic.

11. UI states

State	Expected behavior
Loading	Show a loading indicator while sample data and the first screen load; block double submission during a transfer
Empty	Explain how to create a first account; offer to load sample data
Success	Confirm that an account was created, a transfer completed, or a statement exported, with the new balances
Validation Error	Identify the invalid field (for example an amount of 0 or the same from and to account) and how to fix it
Insufficient Funds	Explain that the transfer would overdraw the account and show the available balance; no balance changes
Unauthorized	Explain that only admins can change overdraft settings or reset data; offer safe navigation
Partial	Show available transactions and a note if some entries could not be read
No Search Results	Show a clear message and a button to reset filters
Storage Error	Explain that data could not be saved and keep the current form on screen

12. Responsiveness

Device	Layout	Expected behavior
Phone	Single column; account list as stacked cards; forms full width; statement table scrolls horizontally	Inputs and Transfer button large enough to tap; no page-level horizontal scrolling
Tablet	Two columns: account list beside the transfer form or statement	List and form visible together
Laptop	Sidebar navigation, summary cards in a row, account list and statement side by side	Full create-transfer-statement workflow visible without excessive scrolling

Responsive requirements:

- Use Bootstrap 5 grid, containers and responsive utilities.
- Use the viewport meta tag and mobile-first CSS.
- Allow the statement table to scroll horizontally on narrow screens.
- Right-align amounts and let long names and references wrap.
- Test at phone and laptop widths, including keyboard-only use.

13. Visual design, accessibility and UX copy

- **Visual style:** Clean, trustworthy banking dashboard with clear balances, simple tables and minimal decoration.
- **Palette:** primary #1D4ED8, text #1F2937 on white, credit/success #166534, debit/error #B42318, warning #B54708, page background #F8FAFC.
- **Contrast ratios (WCAG AA needs 4.5:1 for body text):** text on white 14.7:1; white on primary 6.7:1; success on white 7.1:1; error on white 6.6:1; warning on white 5.4:1.
- **Accessibility labels:** every field has a visible label; debits and credits are shown in separate labelled columns, not by color alone; amounts include the currency symbol; transfer results use aria-live; focus is always visible.
- **Copy voice:** clear, precise and reassuring; short action verbs (Create, Transfer, Export, Reset); errors say what happened and what to do next.

Example UX copy:

- Success: "Transfer complete. ₹1,500.00 sent from ACC-1002 to ACC-1005."
- Insufficient funds: "Transfer blocked. ACC-1002 has ₹800.00 available and no overdraft."
- Invalid input: "Enter an amount greater than ₹0.00."
- Same account: "Choose two different accounts to transfer between."
- Empty state: "No accounts yet. Create your first account or load sample data."

14. Tech stack

- **Stack:** HTML, CSS, JavaScript, Bootstrap 5; Python 3.
- **Framework:** Python 3; HTML, CSS, JavaScript.
- **Libraries / tools:** Bootstrap 5, Git, GitHub; Python standard library (json, dataclasses, hashlib for the stretch ledger).
- **Core tech focus:** HTML5, CSS3 (Flexbox, Grid, Bootstrap 5), vanilla JavaScript (DOM, events, forms, fetch, localStorage), Python basics & OOP, Git/GitHub.
- **Stretch tech:** React + Vite (Vite is in class; React is self-study), Chart.js or Leaflet, or a tiny Python Flask / JSON-file helper.
- **Deployment:** GitHub Pages or Vercel, or a Python CLI demo. Implementation note: all rules run on the client or in a local script for learning only; real banking needs a secure backend, database transactions and audits, and this must be stated in the README.

15. Learning goals

- **Course covered:** Introduction to Computer Programming; Frontend Development Basics.
- **Core learning goals:**
    - Build accessible HTML forms and semantic pages.
    - Style responsive layouts with CSS, Flexbox/Grid and Bootstrap 5.

    - Use JavaScript for events, DOM updates, validation and business rules.
    - Model accounts and transactions with Python classes (OOP) and read and write JSON files.
    - Calculate balances correctly using whole-number amounts and test edge cases.
    - Persist sample data with localStorage and handle storage errors.
    - Use Git/GitHub with clean commits and write a clear README.
- **Stretch learning goals:**
    - Build a hash-chained ledger and explain how it detects tampering (DSA).
    - Build the interface with React + Vite, or plot balances over time with Chart.js.

16. Milestones

Phase	Catalog plan	Expected output
Month 1 — Week 1	One-page brief, Git repo, Bootstrap layout, and one working screen of "CoreBanking Sandbox" with the localStorage/JSON shape sketched	Brief, repository, base layout, documented data shape
Month 1 — Week 2	Must-have flows (Create account; transfer; statement table); form validation; persist sample data. Python helper only if the stack lists it	Working accounts, transfers and statements with overdraft rule
Month 1 — Week 3	Bugfixes, README, GitHub Pages or Vercel demo. Stretch is optional and only if the core flow is solid	Tested core flow, Python CLI helper, README, deployed demo
Month 1 — Week 4	Usability test, accessibility and responsive checks, final polish, demo video or live demo	Final release, usability notes, demo
Month 2	Not applicable: the program duration is 4 weeks.	—
Month 3	Not applicable: the program duration is 4 weeks.	—

17. Evaluation

Catalog metrics:

- Core user flow completes with sample data.
- Role checks block unauthorized actions (for example, only admins can change overdraft settings or reset data).
- Critical-path test or scripted demo checklist passes.
- UI usable on laptop and a phone-width view.
- No account goes negative without an overdraft flag, and every statement's closing balance matches the account balance in the documented test cases.

Suggested evaluation rubric

Evaluation area	Weight
Create account, transfer and statement flows working end to end	30%
Correct balances, transfer rules and overdraft handling	20%
Responsive design, accessibility and UX feedback	15%
Data persistence, validation and Python helper	10%
Testing, code quality and error handling	15%
Documentation and final demonstration	10%
Total	100%

Expected output: Working CoreBanking Sandbox demo on the Sem 1 stack for FinTech Developers, plus README, sample data, and GitHub Pages or Vercel (or a Python CLI demo).

18. Risks and assumptions

Risk / assumption	Likelihood and impact	Mitigation
Floating-point rounding gives wrong balances	Medium; statements do not add up	Store amounts as whole paise and test with hand-calculated cases
Transfer saves the debit but not the credit	Medium; money created or lost	Save both entries in one write and verify totals after each transfer
Negative balance allowed without an overdraft flag	Medium; core rule broken	Check the rule in one TransferService function and test edge cases
Sandbox mistaken for real banking software	Medium; overstated security	Label as demo-only and state limits in the About screen and README
JavaScript and Python rules drift apart	Low; inconsistent results	Document the rules once and run the same test cases in both
localStorage cleared or full	Low; sample data lost	Handle storage errors and provide a sample-data reset

19. Deliverables

- Wireframes for every screen
- GitHub repo with clean commits
- Deployed demo (GitHub Pages / Vercel)
- README with screenshots at phone and laptop width
- Usability test notes
- Demo video or live demo

20. References

- **Reference docs:** MDN Web Docs (HTML, CSS, JS, fetch, localStorage); Python 3 tutorial (files, dicts, classes); Bootstrap 5 docs; GitHub Pages or Vercel getting started.

21. Sign-off

Role	Name	Date	Approved
Student 1	Kunal Tayde		
Student 2	Gajangi Nikhileshwar		
Mentor / Reviewer	Madhushri Tangali		